

Arquitetura de Integração e Arquitetura de Solução

Trilha técnica do básico ao nível sênior para projetar integrações, soluções distribuídas, APIs, eventos, mensageria, governança, segurança, observabilidade e decisões arquiteturais.

Tema	Arquitetura de Integração e Arquitetura de Solução
Nível	Básico → Intermediário → Avançado → Sênior
Foco	Sistemas distribuídos, integração corporativa, APIs, eventos e cloud
Formato	Documento técnico estruturado com sumário, seções, listas e blocos de código

Como usar este documento

Estude cada seção em ordem progressiva, transforme os padrões em pequenos projetos e documente decisões com ADRs/RFCs. O objetivo é sair do conhecimento conceitual para capacidade de desenho, implementação, operação e defesa de trade-offs.

Sumário

1. Visão geral da trilha	5
2. Diferença entre Arquitetura de Solução e Arquitetura de Integração	5
3. Níveis de evolução	7
4. Fundamentos obrigatórios	9
4.1 Requisitos funcionais	9
4.2 Requisitos não funcionais	10
4.3 Restrições arquiteturais	11
5. Papel do Arquiteto de Solução	11
6. Papel do Arquiteto de Integração	12
7. Estilos de integração	13
7.1 Integração por API síncrona	13
7.2 Integração assíncrona por fila	14
7.3 Integração por eventos	15
7.4 Integração batch	17
7.5 Integração por CDC	18
7.6 Integração por banco compartilhado	18
7.7 Integração por arquivos	19
8. Contratos de integração	19
8.1 OpenAPI	20
8.2 AsyncAPI	20
8.3 CloudEvents	21
8.4 Schema Registry	21
9. Padrões de integração essenciais	21
9.1 Message Channel	21
9.2 Message Router	22
9.3 Message Translator	22
9.4 Content Enricher	22
9.5 Content Filter	22
9.6 Splitter	23
9.7 Aggregator	23
9.8 Claim Check	23

9.9 Idempotent Receiver	23
9.10 Dead Letter Channel	23
10. Padrões de arquitetura distribuída	23
10.1 API Gateway	24
10.2 BFF — Backend for Frontend	24
10.3 Circuit Breaker	24
10.4 Retry com backoff	25
10.5 Bulkhead	25
10.6 Saga	25
10.7 Outbox Pattern	25
10.8 Inbox Pattern	26
10.9 CQRS	26
10.10 Event Sourcing	26
11. Arquitetura orientada a eventos	26
12. Arquitetura de APIs	28
13. Arquitetura de dados em integração	30
14. Segurança em arquitetura de solução e integração	31
15. Observabilidade de integração	32
16. Resiliência	33
17. Governança de arquitetura	34
18. Documentação arquitetural	35
1. Diagrama de contexto	36
2. Diagrama de containers	36
3. Diagrama de componentes	36
4. Diagrama de sequência	36
5. Diagrama de integração	36
6. Diagrama de deployment	37
7. Diagrama de dados	37
19. ADR — Architecture Decision Record	37
20. RFC técnico	38
21. Arquitetura cloud para solução e integração	39

22. Ferramentas e tecnologias importantes	40
23. Trilha de estudos por fases	41
Fase 1 — Fundamentos de arquitetura	41
24. Projeto prático principal da trilha	45
25. Checklist de nível sênior	47
26. Erros comuns que impedem senioridade	47
27. Livros e referências recomendadas	48
28. Resumo final da trilha	48

1. Visão geral da trilha

Objetivo

Formar domínio para atuar como:

- Desenvolvedor backend com visão arquitetural.
- Arquiteto de integração.
- Arquiteto de solução.
- Tech lead.
- Solution designer.
- Staff engineer em sistemas distribuídos.
- Arquiteto cloud/backend com foco em integração corporativa.
- O que você precisa saber ao final

Você deve ser capaz de:

- Entender requisitos de negócio e traduzi-los em arquitetura.
- Desenhar soluções ponta a ponta.
- Integrar sistemas legados, APIs, bancos, filas, eventos, SaaS e cloud.
- Escolher entre REST, gRPC, mensageria, eventos, ETL, CDC e batch.
- Projetar APIs síncronas e assíncronas.
- Definir contratos com OpenAPI, AsyncAPI e schemas de eventos.
- Desenhar fluxos de integração com resiliência, segurança e observabilidade.
- Aplicar padrões como API Gateway, BFF, Saga, Outbox, Inbox, CQRS, Event Sourcing, Circuit Breaker e Idempotency.
- Criar diagramas C4, sequência, deployment, integração e dados.
- Documentar decisões com ADRs e RFCs.
- Defender trade-offs técnicos para negócio, produto, segurança, infraestrutura e desenvolvimento.

2. Diferença entre Arquitetura de Solução e Arquitetura de Integração

Arquitetura de Solução

Arquitetura de solução responde:

- “Qual solução técnica atende a este problema de negócio, considerando requisitos funcionais, não funcionais, restrições, custo, segurança, operação e evolução?”

Ela cobre:

- Escopo da solução.
- Sistemas envolvidos.
- Componentes.
- APIs.
- Banco de dados.

- Segurança.
- Infraestrutura.
- Cloud.
- Observabilidade.
- Deploy.
- Integrações.
- Escalabilidade.
- Custos.
- Riscos.
- Roadmap técnico. O Azure Architecture Center define arquitetura como o desenho de workloads por meio da integração de serviços de plataforma, funcionalidades e código para atender requisitos funcionais e não funcionais.

Arquitetura de Integração

Arquitetura de integração responde:

- “Como sistemas diferentes trocam dados, comandos, eventos e estados de maneira segura, resiliente, governada e escalável?”

Ela cobre:

- APIs.
- Eventos.
- Filas.
- Tópicos.
- ETL/ELT.
- CDC.
- Webhooks.
- Batch.
- Streaming.
- Contratos.
- Schemas.
- Transformações.
- Roteamento.
- Orquestração.
- Coreografia.
- Idempotência.
- Reprocessamento.
- Monitoramento.
- Versionamento.
- Governança. A Microsoft define o propósito da integração como conectar aplicações, dados, serviços e dispositivos, muitas vezes em cenários complexos, para unir workflows de forma consistente e escalável.

3. Níveis de evolução

Nível básico

Foco:

- Entender sistemas, APIs, bancos, integrações simples e diagramas básicos.

Você deve aprender:

- O que é arquitetura.
- O que é requisito funcional.
- O que é requisito não funcional.
- O que é API.
- O que é integração.
- O que é contrato.
- O que é sistema legado.
- O que é fluxo síncrono.
- O que é fluxo assíncrono.
- O que é fila.
- O que é evento.
- O que é webhook.
- O que é batch.
- O que é banco compartilhado.
- O que é acoplamento.
- O que é coesão.
- O que é latência.
- O que é disponibilidade.
- O que é escalabilidade.

Resultado esperado:

- Conseguir desenhar um fluxo simples entre frontend, backend, banco e API externa.
- Nível intermediário

Foco:

- Projetar integrações reais com APIs, mensageria, autenticação, tratamento de erros e documentação.

Você deve aprender:

- REST.
- OpenAPI.
- HTTP status.
- OAuth2/OIDC.
- API Gateway.
- Mensageria.
- RabbitMQ/Kafka/SQS.
- Retry.

- Timeout.
- DLQ.
- Idempotência.
- Versionamento de API.
- Contratos.
- Logs.
- Métricas.
- Tracing.
- Diagramas C4.
- Diagramas de sequência.
- ADRs.

Resultado esperado:

- Conseguir desenhar e implementar uma integração entre microserviços com contrato, segurança, reprocessamento e observabilidade.
- Nível avançado

Foco:

- Arquiteturas distribuídas, eventos, consistência, governança e cloud.

Você deve aprender:

- Event-driven architecture.
- Kafka.
- EventBridge.
- Schema Registry.
- Avro/Protobuf/JSON Schema.
- Saga.
- Outbox.
- Inbox.
- CDC.
- CQRS.
- Event Sourcing.
- API Management.
- Service Mesh.
- BFF.
- Backend for Frontend.
- Anti-corruption layer.
- Canonical Data Model.
- Domain events.
- Integration events.
- Data contracts.
- Observabilidade distribuída.
- Segurança em APIs/eventos.

- Well-Architected Framework.
- Arquitetura multi-cloud/híbrida.

Resultado esperado:

- Conseguir desenhar uma arquitetura de integração corporativa com múltiplos sistemas, eventos, APIs, consistência eventual e governança.
- Nível sênior

Foco:

- Decisão arquitetural, estratégia, governança, risco, custo, operação, evolução e comunicação executiva.

Você deve dominar:

- Trade-offs entre integração síncrona, assíncrona, batch, streaming e CDC.
- Consistência distribuída.
- CAP/PACELC.
- Resiliência sistêmica.
- Degradação controlada.
- Arquitetura multi-região.
- Integração com legados.
- Estratégia de migração.
- Governança de APIs.
- Governança de eventos.
- Catálogo de integrações.
- Segurança ponta a ponta.
- Observabilidade cross-system.
- RPO/RTO.
- FinOps.
- Risk assessment.
- ADRs executivos.
- RFCs técnicos.
- Arquitetura evolutiva.
- Arquitetura orientada a capacidades de negócio.

Resultado esperado:

- Conseguir liderar a arquitetura de uma solução corporativa de ponta a ponta e defender decisões para times técnicos, segurança, produto, negócio e liderança.

4. Fundamentos obrigatórios

4.1 Requisitos funcionais

Requisitos funcionais descrevem o que o sistema precisa fazer.

Exemplos:

- Criar pedido.

- Consultar cliente.
- Processar pagamento.
- Emitir nota fiscal.
- Enviar notificação.
- Gerar relatório.
- Importar arquivo.
- Sincronizar cadastro.
- Publicar evento de venda.
- Consultar estoque.

Um arquiteto precisa transformar requisitos funcionais em:

- Casos de uso.
- Serviços.
- APIs.
- Eventos.
- Dados.
- Fluxos.
- Regras de negócio.
- Critérios de aceite.

4.2 Requisitos não funcionais

Requisitos não funcionais definem como o sistema deve se comportar.

Principais categorias:

- Disponibilidade.
- Escalabilidade.
- Performance.
- Latência.
- Segurança.
- Privacidade.
- Observabilidade.
- Resiliência.
- Manutenibilidade.
- Testabilidade.
- Portabilidade.
- Custo.
- Compliance.
- Recuperação de desastre.
- RPO.
- RTO. AWS Well-Architected organiza boas práticas de arquitetura em seis pilares: excelência operacional, segurança, confiabilidade, eficiência de performance, otimização de custo e sustentabilidade.

Critério sênior:

- Nunca desenhar solução apenas com base em funcionalidade.
- Toda arquitetura séria precisa declarar requisitos não funcionais e trade-offs.

4.3 Restrições arquiteturais

Restrições são limites impostos ao desenho.

Exemplos:

- Precisa usar Java/Spring.
- Precisa rodar na AWS.
- Precisa integrar com sistema legado.
- Precisa manter banco atual.
- Precisa cumprir LGPD.
- Precisa entregar em 3 meses.
- Precisa evitar alto custo.
- Precisa suportar 1.000 requisições por segundo.
- Precisa tolerar indisponibilidade de parceiro.
- Precisa manter compatibilidade com clientes antigos.

Critério sênior:

- Saber diferenciar requisito de desejo.
- Saber separar restrição real de preferência técnica.

5. Papel do Arquiteto de Solução

Responsabilidades**Um arquiteto de solução precisa:**

- Entender problema de negócio.
- Mapear stakeholders.
- Levantar requisitos.
- Identificar sistemas envolvidos.
- Definir fronteiras.
- Escolher padrões.
- Escolher tecnologias.
- Avaliar riscos.
- Desenhar solução lógica.
- Desenhar solução física.
- Definir integrações.
- Validar segurança.
- Validar operação.
- Validar custo.
- Validar evolução.

- Produzir documentação.
- Acompanhar implementação.
- Revisar aderência arquitetural.
- Entregáveis

Entregáveis típicos:

- Visão de solução.
- Diagrama de contexto.
- Diagrama de containers.
- Diagrama de componentes.
- Diagrama de sequência.
- Diagrama de integração.
- Diagrama de implantação.
- Modelo de dados.
- Contratos de API.
- Contratos de eventos.
- Matriz de requisitos não funcionais.
- Matriz de riscos.
- ADRs.
- RFC.
- Plano de migração.
- Plano de rollback.
- Estratégia de observabilidade.
- Estratégia de segurança.
- Estratégia de testes.
- Estimativa técnica.
- Roadmap técnico. O C4 Model foi criado para ajudar times de software a descrever e comunicar arquitetura em diferentes níveis de detalhe, como “mapas do código”.

6. Papel do Arquiteto de Integração

Responsabilidades**Um arquiteto de integração precisa:**

- Mapear sistemas produtores e consumidores.
- Definir estilo de integração.
- Definir contratos.
- Definir schemas.
- Definir protocolos.
- Definir autenticação.
- Definir autorização.
- Definir transformação de dados.

- Definir roteamento.
- Definir tratamento de erro.
- Definir retry.
- Definir DLQ.
- Definir reprocessamento.
- Definir idempotência.
- Definir correlação.
- Definir rastreabilidade.
- Definir versionamento.
- Definir governança.
- Definir monitoração.
- Perguntas que ele precisa responder
- A integração deve ser síncrona ou assíncrona?
- O consumidor precisa de resposta imediata?
- O produtor pode esperar?
- Qual é a tolerância a atraso?
- Qual é a volumetria?
- O dado precisa ser exatamente consistente?
- Pode haver consistência eventual?
- Como evitar duplicidade?
- Como tratar falha parcial?
- Como reprocessar?
- Como auditar?
- Como versionar?
- Como proteger dados sensíveis?
- Como monitorar ponta a ponta?
- Como desligar ou migrar a integração no futuro?

7. Estilos de integração

7.1 Integração por API síncrona

Usada quando um sistema precisa de resposta imediata.

Exemplos:

- Consultar CPF.
- Validar cliente.
- Buscar preço.
- Consultar estoque.
- Autenticar usuário.
- Simular frete.
- Aprovar cartão em tempo real.

Tecnologias:

- REST.
- GraphQL.
- gRPC.
- SOAP em legados.
- API Gateway.
- BFF.
- Service Mesh.

Vantagens:

- Simples de entender.
- Resposta imediata.
- Boa para consultas.
- Boa para interações de usuário.

Riscos:

- Acoplamento temporal.
- Cascata de falhas.
- Timeouts.
- Latência.
- Dependência direta.
- Menor resiliência se mal desenhada.

Padrões obrigatórios:

- Timeout.
- Retry com backoff.
- Circuit breaker.
- Bulkhead.
- Rate limit.
- Fallback.
- Idempotency key.
- Correlation ID.
- Observabilidade.

7.2 Integração assíncrona por fila

Usada quando o produtor não precisa esperar o processamento final.

Exemplos:

- Enviar e-mail.
- Processar imagem.
- Gerar relatório.
- Emitir nota fiscal.
- Processar pagamento em segunda etapa.
- Integrar pedido com ERP.

- Registrar auditoria.

Tecnologias:

- RabbitMQ.
- AWS SQS.
- Azure Service Bus.
- Google Pub/Sub.
- ActiveMQ.
- IBM MQ.

Vantagens:

- Desacopla produtor e consumidor.
- Melhora resiliência.
- Absorve picos.
- Permite reprocessamento.
- Reduz falha em cascata.

Riscos:

- Consistência eventual.
- Duplicidade.
- Ordem.
- DLQ mal gerida.
- Reprocessamento incorreto.
- Falta de rastreabilidade.

Padrões obrigatórios:

- DLQ.
- Retry policy.
- Idempotent consumer.
- Poison message handling.
- Correlation ID.
- Message expiration.
- Message version.
- Monitoring de fila.
- Backpressure.

7.3 Integração por eventos

Usada quando mudanças de estado precisam ser publicadas para múltiplos consumidores.

Exemplos:

- OrderCreated.
- PaymentApproved.
- InvoiceIssued.
- CustomerUpdated.
- ProductPriceChanged.

- StockReserved.
- UserRegistered.

Tecnologias:

- Kafka.
- EventBridge.
- SNS.
- Azure Event Grid.
- Google Pub/Sub.
- Pulsar.
- NATS. Apache Kafka é uma plataforma de event streaming usada para publicar, assinar, armazenar e processar streams de eventos.

Vantagens:

- Desacoplamento forte.
- Escala bem.
- Permite múltiplos consumidores.
- Facilita auditoria.
- Facilita analytics.
- Facilita integração em tempo real.

Riscos:

- Governança de eventos.
- Evolução de schema.
- Ordem de eventos.
- Duplicidade.
- Reprocessamento.
- Contratos fracos.
- Eventos grandes demais.
- Eventos sem semântica de negócio.

Padrões obrigatórios:

- Event naming.
- Event schema.
- Schema Registry.
- Event versioning.
- Idempotent consumer.
- Outbox.
- Inbox.
- Consumer lag monitoring.
- Replay strategy.
- Dead-letter topic.
- Event catalog.

7.4 Integração batch

Usada quando dados podem ser processados em janelas.

Exemplos:

- Fechamento financeiro.
- Conciliação.
- Importação de arquivos.
- Relatórios.
- Sincronização diária.
- Carga para data warehouse.
- Rotinas fiscais.

Tecnologias:

- Arquivos CSV/JSON/XML.
- SFTP.
- Spring Batch.
- Airflow.
- Glue.
- Data Factory.
- dbt.
- Jobs Kubernetes.
- CronJobs.

Vantagens:

- Simples para grandes volumes.
- Boa para legados.
- Boa para fechamento e conciliação.
- Menos dependente de disponibilidade em tempo real.

Riscos:

- Latência alta.
- Falha tardia.
- Duplicidade.
- Arquivo corrompido.
- Reprocessamento complexo.
- Baixa rastreabilidade se mal feito.

Padrões obrigatórios:

- Checkpoint.
- Job id.
- Controle de lote.
- Idempotência.
- Validação de arquivo.
- Quarentena.

- Reprocessamento.
- Auditoria.
- Relatório de inconsistências.

7.5 Integração por CDC

CDC significa Change Data Capture. Ele captura mudanças no banco e publica essas mudanças para outros sistemas.

Exemplos:

- PostgreSQL → Debezium → Kafka.
- MySQL binlog → Kafka.
- MongoDB change streams → consumidores.
- Outbox table → Debezium → eventos.

Vantagens:

- Reduz acoplamento no código.
- Boa para integração com legado.
- Boa para sincronização de dados.
- Boa para eventos derivados do banco.

Riscos:

- Expor modelo interno.
- Contrato fraco se publicar tabela crua.
- Ordem e consistência.
- Evolução de schema.
- Dependência do banco.
- Complexidade operacional.

Uso sênior:

- Preferir CDC com Outbox para publicar eventos de domínio/integração, não simplesmente replicar tabela interna sem governança.

7.6 Integração por banco compartilhado

Vários sistemas acessam o mesmo banco.

Vantagens:

- Simples no começo.
- Evita criar API inicialmente.
- Pode ajudar em legados.

Riscos:

- Altíssimo acoplamento.
- Quebra encapsulamento.
- Dificulta evolução.
- Dificulta segurança.
- Dificulta auditoria.

- Um sistema pode quebrar outro.
- Modelo de dados vira contrato accidental.

Critério sênior:

- Evitar como padrão.
- Aceitar temporariamente em legado, com plano de migração.

7.7 Integração por arquivos

Muito comum em bancos, ERPs, fiscais e legados.

Exemplos:

- CSV.
- XML.
- JSON.
- CNAB.
- EDI.
- OFX.
- Remessa/retorno.
- SFTP.
- Object Storage.

Riscos:

- Layout divergente.
- Encoding.
- Duplicidade.
- Reprocessamento.
- Arquivo parcial.
- Falta de rastreabilidade.
- Baixa governança.

Padrões obrigatórios:

- Nome padronizado.
- Layout versionado.
- Checksum.
- Controle de lote.
- Pasta de entrada/processado/erro.
- Auditoria.
- Idempotência.
- Relatório de rejeição.

8. Contratos de integração

8.1 OpenAPI

OpenAPI é usado para descrever APIs HTTP de forma padronizada, permitindo que humanos e ferramentas entendam endpoints, parâmetros, respostas, autenticação e outros detalhes sem precisar inspecionar o código.

Dominar:

- Paths.
- Operations.
- Parameters.
- Request body.
- Responses.
- Schemas.
- Security schemes.
- Examples.
- Error model.
- Versioning.
- Code generation.
- Contract testing.
- API linting.

Critério sênior:

- API sem OpenAPI em ambiente corporativo é dívida de governança.

8.2 AsyncAPI

AsyncAPI é uma especificação voltada para arquiteturas orientadas a eventos e APIs assíncronas, ajudando a definir e manter contratos de mensagens.

Dominar:

- Channels.
- Messages.
- Schemas.
- Publishers.
- Subscribers.
- Bindings.
- Kafka binding.
- AMQP binding.
- WebSocket binding.
- Examples.
- Versioning.
- Event documentation.

Critério sênior:

- Eventos precisam de contrato tanto quanto APIs REST.

8.3 CloudEvents

CloudEvents é uma especificação para descrever dados de eventos de forma comum, facilitando interoperabilidade entre serviços, plataformas e sistemas.

Dominar campos como:

- id.
- source.
- type.
- specversion.
- subject.
- time.
- datacontenttype.
- data.

Critério sênior:

- Usar envelope padronizado para eventos facilita roteamento, auditoria, rastreabilidade e integração multi-plataforma.

8.4 Schema Registry

Dominar:

- Avro.
- JSON Schema.
- Protobuf.
- Backward compatibility.
- Forward compatibility.
- Full compatibility.
- Breaking changes.
- Schema evolution.
- Subject naming strategy.
- Compatibility checks no pipeline.

Critério sênior:

- Evento sem schema versionado vira risco operacional.

9. Padrões de integração essenciais

O catálogo Enterprise Integration Patterns reúne 65 padrões de integração baseados em experiência prática de projetos e continua sendo uma referência central para soluções de mensageria e integração robusta.

9.1 Message Channel

Canal por onde mensagens trafegam.

Exemplos:

- Queue.

- Topic.
- Stream.
- Event bus.

Uso:

- Separar produtores e consumidores.

9.2 Message Router

Roteia mensagens para destinos diferentes.

Variações:

- Content-Based Router.
- Dynamic Router.
- Recipient List.
- Routing Slip.

Uso:

- Enviar eventos para fluxos diferentes dependendo do conteúdo.

9.3 Message Translator

Transforma formato de mensagem.

Exemplos:

- XML → JSON.
- Legacy DTO → canonical model.
- ERP payload → domínio interno.

Uso:

- Evitar contaminar domínio interno com modelo externo.

9.4 Content Enricher

Enriquece mensagem com dados adicionais.

Exemplo:

- Evento de pedido contém customerId.
- Enricher consulta cliente e adiciona segmento, região e score.

Risco:

- Pode introduzir latência e dependência.

9.5 Content Filter

Remove dados desnecessários ou sensíveis.

Uso:

- Evitar vazamento de dados.
- Reduzir payload.
- Ajustar mensagem para consumidor.

9.6 Splitter

Divide mensagem grande em partes menores.

Exemplo:

- Um arquivo com 10.000 registros vira 10.000 mensagens.

9.7 Aggregator

Junta múltiplas mensagens em uma visão consolidada.

Exemplo:

- Pedido + pagamento + nota + entrega → visão final do pedido.

9.8 Claim Check

Guarda payload grande em storage e trafega apenas referência.

Exemplo:

- Mensagem carrega documentUrl, não o PDF inteiro.

9.9 Idempotent Receiver

Consumidor que processa a mesma mensagem mais de uma vez sem gerar duplicidade.

Obrigatório para:

- Filas.
- Kafka.
- Webhooks.
- Pagamentos.
- Pedidos.
- Integrações fiscais.

9.10 Dead Letter Channel

Canal para mensagens que não puderam ser processadas.

Obrigatório ter:

- Motivo do erro.
- Tentativas.
- Timestamp.
- Payload original.
- Correlation ID.
- Estratégia de reprocessamento.

10. Padrões de arquitetura distribuída

10.1 API Gateway

API Gateway centraliza entrada para múltiplos serviços, podendo tratar autenticação, roteamento, rate limiting, agregação e políticas transversais. O padrão aparece como necessário em arquiteturas de microsserviços para rotear chamadas aos serviços e aplicar preocupações transversais.

Usar para:

- Roteamento.
- Autenticação.
- Rate limit.
- TLS.
- Logging.
- Transformação.
- Versões.
- Throttling.

Cuidado:

- Não transformar o gateway em “monólito de regras de negócio”.

10.2 BFF — Backend for Frontend

Usar quando:

- Web, mobile e parceiros têm necessidades diferentes.
- O frontend precisa de agregação.
- O backend de domínio não deve ser exposto diretamente.
- É necessário otimizar payload por canal.

Exemplo:

- bff-web.
- bff-mobile.
- bff-admin.

10.3 Circuit Breaker

Evita que falhas repetidas em serviço externo derrubem o sistema chamador.

Estados:

- Closed.
- Open.
- Half-open.

Usar com:

- Timeout.
- Retry limitado.
- Fallback.
- Métricas.

10.4 Retry com backoff

Usar para falhas transitórias.

Cuidado:

- Retry sem limite causa tempestade.
- Retry em operação não idempotente pode duplicar efeito.
- Retry precisa de backoff e jitter.

10.5 Bulkhead

Isola recursos para evitar que uma falha consuma tudo.

Exemplo:

- Pool separado para pagamento.
- Pool separado para consulta de cliente.
- Thread pool separado por integração crítica.

10.6 Saga

Saga é uma sequência de transações locais; cada etapa atualiza seu próprio banco e publica mensagem/evento para a próxima etapa. Se uma etapa falha por regra de negócio, são executadas transações compensatórias para desfazer efeitos anteriores.

Tipos:

- Orquestrada.
- Coreografada.

Usar para:

- Pedido.
- Pagamento.
- Estoque.
- Entrega.
- Nota fiscal.

Cuidado:

- Compensação nem sempre é simples.
- Estado da saga precisa ser auditável.
- Eventos precisam ser idempotentes.

10.7 Outbox Pattern

Problema:

- Salvar no banco e publicar evento são duas operações diferentes.
- Uma pode funcionar e outra falhar.

Solução:

- Salvar mudança de domínio e evento em uma tabela outbox na mesma transação.
- Publicador lê outbox e envia ao broker.
- Marca como publicado.

Uso:

- Garantir publicação confiável de evento após mudança no banco.

10.8 Inbox Pattern

Problema:

- Consumidor pode receber mensagem duplicada.

Solução:

- Registrar mensagens processadas em tabela inbox.
- Ignorar mensagens já processadas.

Uso:

- Consumidores idempotentes.

10.9 CQRS

Separa modelo de escrita e leitura.

Usar quando:

- Leitura e escrita têm necessidades muito diferentes.
- Consultas são complexas.
- Performance de leitura precisa ser otimizada.
- Há projeções derivadas de eventos.

Cuidado:

- Aumenta complexidade.
- Nem todo CRUD precisa de CQRS.

10.10 Event Sourcing

Armazena eventos como fonte da verdade, não apenas estado final.

Usar quando:

- Auditoria completa é obrigatória.
- Histórico de mudanças é essencial.
- Estado pode ser reconstruído.
- Domínio é altamente orientado a eventos.

Cuidado:

- Evolução de eventos é difícil.
- Queries exigem projeções.
- Debug exige maturidade.

11. Arquitetura orientada a eventos

Conceitos centrais**Dominar:**

- Evento.

- Comando.
- Query.
- Notificação.
- Fato de negócio.
- Event storming.
- Event bus.
- Broker.
- Topic.
- Partition.
- Consumer group.
- Offset.
- Replay.
- Ordering.
- Delivery semantics.
- At-most-once.
- At-least-once.
- Exactly-once conceitual.
- Idempotência.
- Schema evolution.
- Evento bom vs evento ruim

Evento ruim:

```
{
  "status": "OK",
  "data": "123"
}
```

Evento bom:

```
{
  "id": "evt-123",
  "type": "order.payment.approved.v1",
  "source": "payment-service",
  "time": "2026-06-06T12:00:00Z",
  "subject": "order-987",
  "data": {
    "orderId": "order-987",
    "paymentId": "pay-456",
    "amount": 199.90,
    "currency": "BRL"
  }
}
```

Critérios sênior

Um evento sênior precisa ter:

- Nome semântico.
- Versão.

- Identificador único.
- Origem.
- Timestamp.
- Correlation ID.
- Causation ID.
- Schema.
- Compatibilidade.
- Dono.
- SLA de publicação.
- Política de retenção.
- Estratégia de replay.
- Consumidores conhecidos.
- Classificação de dados sensíveis.

12. Arquitetura de APIs

Conteúdos obrigatórios

Dominar:

- REST.
- HTTP.
- OpenAPI.
- GraphQL.
- gRPC.
- Webhooks.
- WebSocket.
- SSE.
- Versionamento.
- Paginação.
- Filtros.
- Ordenação.
- Idempotency key.
- Rate limit.
- Auth.
- Error model.
- Problem Details.
- API Gateway.
- Developer Portal.
- API Catalog.
- API Lifecycle.
- REST

Usar quando:

- Recursos são bem definidos.
- Operações CRUD/consulta dominam.
- Ecossistema amplo é importante.
- Cache HTTP é útil.
- GraphQL

Usar quando:

- Clientes precisam escolher campos.
- Há múltiplas fontes agregadas.
- Frontend precisa reduzir overfetch/underfetch.

Cuidado:

- Autorização por campo.
- Complexidade de query.
- N+1.
- Cache mais difícil.
- Observabilidade mais difícil.
- gRPC

Usar quando:

- Comunicação interna de baixa latência.
- Contrato forte.
- Streaming.
- Alto volume.
- Polyglot com Protobuf.

Cuidado:

- Debug menos simples.
- Browser exige cuidado.
- Menos amigável para APIs públicas tradicionais.
- Webhook

Usar quando:

- Sistema externo precisa ser notificado.
- Integração B2B.
- Pagamento.
- Antifraude.
- ERP.
- SaaS.

Obrigatório:

- Assinatura.
- Retry.
- Idempotência.

- Event ID.
- Timestamp.
- Secret rotation.
- Logs.
- Reenvio manual.

13. Arquitetura de dados em integração

Tópicos essenciais

Dominar:

- Modelo canônico.
- Modelo por domínio.
- Data ownership.
- Data contracts.
- Master data.
- Reference data.
- Transactional data.
- Analytical data.
- Data lineage.
- Data quality.
- Data governance.
- LGPD.
- Mascaramento.
- Criptografia.
- Retenção.
- Purge.
- Auditoria.
- Modelo canônico

Útil quando:

- Muitos sistemas precisam falar uma linguagem comum.
- Há integração corporativa centralizada.
- Existem muitos modelos legados divergentes.

Risco:

- Pode virar modelo gigante e burocrático.
- Pode acoplar todos a um “supercontrato”.
- Modelo por domínio

Útil quando:

- A organização trabalha com DDD.
- Cada domínio possui linguagem própria.
- Eventos são publicados com semântica do domínio.

Critério sênior:

- Usar modelo canônico para integração corporativa quando fizer sentido, mas evitar que ele destrua a autonomia dos domínios.

14. Segurança em arquitetura de solução e integração

Fundamentos

Dominar:

- Autenticação.
- Autorização.
- OAuth2.
- OIDC.
- JWT.
- Scopes.
- Claims.
- RBAC.
- ABAC.
- mTLS.
- API keys.
- Secrets.
- KMS.
- Criptografia em trânsito.
- Criptografia em repouso.
- Assinatura de webhooks.
- Token rotation.
- Least privilege.
- Zero Trust.
- Threat modeling.
- LGPD.
- Segurança em APIs

Aplicar:

- API Gateway.
- Rate limit.
- WAF.
- CORS correto.
- Validação de input.
- Output encoding.
- Proteção contra mass assignment.
- Proteção contra BOLA/IDOR.

- Proteção contra SSRF.
- Logs sem dados sensíveis.
- Error messages seguras.
- Segurança em eventos

Aplicar:

- Controle de acesso por tópico.
- Criptografia.
- Schema sem dados sensíveis desnecessários.
- Mascaramento.
- Auditoria.
- Retenção adequada.
- Segregação por domínio.
- Política de consumidores.
- DLQ protegida.

Critério sênior:

- Segurança não pode ser “camada final”; precisa estar no desenho inicial.

15. Observabilidade de integração

O que medir**Métricas técnicas:**

- Taxa de erro.
- Latência.
- Throughput.
- Lag.
- Tamanho de fila.
- Tempo de processamento.
- Retries.
- DLQ.
- Timeouts.
- Circuit breaker open.
- Consumer failures.
- Schema validation failures.

Métricas de negócio:

- Pedidos criados.
- Pagamentos aprovados.
- Notas emitidas.
- Integrações pendentes.
- Arquivos processados.
- Registros rejeitados.

- Tempo até conclusão do fluxo.
- Taxa de sucesso ponta a ponta.
- Logs

Todo fluxo deve ter:

- Correlation ID.
- Causation ID.
- User ID quando aplicável.
- Tenant ID quando aplicável.
- Event ID.
- Request ID.
- System origin.
- System destination.
- Status.
- Error code.
- Business key.
- Tracing

Aplicar:

- OpenTelemetry.
- Distributed tracing.
- Propagação de trace em HTTP.
- Propagação de trace em mensagens.
- Span por integração.
- Span por chamada externa.
- Span por processamento assíncrono.

Critério sênior:

- Deve ser possível responder: “onde exatamente o fluxo quebrou?” sem abrir 10 sistemas manualmente.

16. Resiliência

Padrões obrigatórios

Dominar:

- Timeout.
- Retry.
- Backoff.
- Jitter.
- Circuit breaker.
- Bulkhead.
- Rate limiter.
- Fallback.

- Queue buffering.
- DLQ.
- Reprocessamento.
- Idempotência.
- Checkpoint.
- Graceful degradation.
- Load shedding.
- Health checks.
- Readiness.
- Liveness.
- Chaos testing.
- Disaster recovery.
- Erros comuns

Evitar:

- Retry infinito.
- Timeout muito alto.
- Sem timeout.
- Fila sem DLQ.
- Consumidor não idempotente.
- API sem idempotency key.
- Erro técnico misturado com erro de negócio.
- DLQ sem processo de reprocessamento.
- Sem métrica de lag.
- Sem correlação ponta a ponta.
- Sem plano de rollback.

17. Governança de arquitetura

Governança de APIs**Definir:**

- Padrão de URL.
- Padrão de erro.
- Padrão de paginação.
- Padrão de versionamento.
- Padrão de autenticação.
- Padrão de rate limit.
- OpenAPI obrigatório.
- Lint de contrato.
- Contract testing.
- Deprecation policy.

- API catalog.
- Owner.
- SLA.
- SLO.
- Documentação.
- Governança de eventos

Definir:

- Naming convention.
- Schema obrigatório.
- AsyncAPI.
- CloudEvents.
- Versionamento.
- Compatibilidade.
- Retenção.
- Dono.
- Consumidores.
- LGPD.
- Replay.
- DLQ.
- Observabilidade.
- Event catalog.
- Governança de decisões

Definir:

- ADR.
- RFC.
- Architecture Review.
- Security Review.
- Cost Review.
- Operational Readiness Review.
- Threat Modeling.
- Production Readiness Checklist.

Critério sênior:

- Arquitetura sem governança vira coleção de decisões locais conflitantes.

18. Documentação arquitetural

Diagramas obrigatórios

1. Diagrama de contexto

Mostra:

- Usuários.
- Sistemas externos.
- Sistema principal.
- Relações de alto nível.

2. Diagrama de containers

Mostra:

- Frontend.
- Backend.
- Banco.
- Cache.
- Broker.
- Serviços externos.
- Infra principal.

3. Diagrama de componentes

Mostra:

- Módulos internos.
- Use cases.
- Adapters.
- Ports.
- Repositories.
- Clients.
- Publishers.
- Consumers.

4. Diagrama de sequência

Mostra:

- Ordem das chamadas.
- Fluxo síncrono.
- Fluxo assíncrono.
- Falhas.
- Alternativas.

5. Diagrama de integração

Mostra:

- Sistemas.
- Protocolos.

- APIs.
- Eventos.
- Filas.
- Transformações.
- Frequência.
- Direção dos dados.

6. Diagrama de deployment

Mostra:

- Cloud.
- VPC/VNet.
- Subnets.
- Load balancer.
- Containers.
- Kubernetes.
- Bancos.
- Broker.
- Secrets.
- Observabilidade.

7. Diagrama de dados

Mostra:

- Entidades.
- Ownership.
- Fluxo de dados.
- Classificação.
- Retenção.
- Replicação.
- CDC.

19. ADR — Architecture Decision Record

Estrutura recomendada

```
# ADR-001 – Usar Kafka para eventos de pedido
## Status
Aceito
## Contexto
O fluxo de pedido precisa notificar pagamento, estoque, nota fiscal e analytics.
## Decisão
Usaremos Kafka com eventos versionados e Schema Registry.
## Consequências positivas
- Desacoplamento entre domínios.
- Replay de eventos.
- Escala horizontal de consumidores.
## Consequências negativas
- Maior complexidade operacional.
- Necessidade de governança de schemas.
- Consistência eventual.
## Alternativas consideradas
- REST síncrono.
- RabbitMQ.
- EventBridge.
## Critérios de aceitação
- Eventos documentados em AsyncAPI.
- Consumidores idempotentes.
- DLQ configurada.
- Métricas de lag.
```

Critério sênior:

- Toda decisão relevante precisa registrar contexto, decisão, alternativas e consequências.

20. RFC técnico

Quando usar

Usar RFC para decisões maiores:

- Nova arquitetura.
- Nova integração crítica.
- Migração de sistema.
- Mudança de contrato.
- Adoção de Kafka.
- Adoção de API Gateway.
- Estratégia multi-cloud.
- Separação de monólito.
- Novo BFF.
- Nova política de autenticação.
- Estrutura
- Resumo.
- Problema.
- Objetivos.
- Não objetivos.
- Contexto atual.
- Proposta.

- Alternativas.
- Trade-offs.
- Impacto em segurança.
- Impacto em operação.
- Impacto em custo.
- Impacto em times.
- Plano de migração.
- Plano de rollback.
- Métricas de sucesso.
- Perguntas em aberto.

21. Arquitetura cloud para solução e integração

AWS

Dominar:

- API Gateway.
- Lambda.
- EventBridge.
- SQS.
- SNS.
- Step Functions.
- ECS.
- EKS.
- RDS.
- DynamoDB.
- ElastiCache.
- MSK.
- CloudWatch.
- X-Ray.
- IAM.
- KMS.
- Secrets Manager.
- WAF.
- CloudFront.
- Route 53.
- Well-Architected.
- Azure

Dominar:

- API Management.

- Functions.
- Logic Apps.
- Event Grid.
- Event Hubs.
- Service Bus.
- Data Factory.
- Application Insights.
- Key Vault.
- AKS.
- SQL Database.
- Cosmos DB.
- Azure Monitor.
- Google Cloud

Dominar:

- Apigee.
- Cloud Run.
- Cloud Functions.
- Pub/Sub.
- Workflows.
- Dataflow.
- GKE.
- Cloud SQL.
- Firestore.
- BigQuery.
- Cloud Monitoring.
- Secret Manager.

Critério sênior:

- Saber escolher serviço por requisito, não por preferência.

22. Ferramentas e tecnologias importantes

APIs

OpenAPI. Swagger. Postman. Insomnia. Stoplight. Kong. Apigee. AWS API Gateway. Azure API Management. NGINX. Envoy.

Mensageria e eventos

Kafka. RabbitMQ. AWS SQS. AWS SNS. AWS EventBridge. Azure Service Bus. Azure Event Grid. Google Pub/Sub. NATS. Pulsar.

Integração corporativa

Apache Camel. Spring Integration. MuleSoft. WSO2. Boomi. Logic Apps. Data Factory.

Contratos

OpenAPI. AsyncAPI. JSON Schema. Avro. Protobuf. CloudEvents. Schema Registry.

Observabilidade

OpenTelemetry. Prometheus. Grafana. Jaeger. Tempo. Loki. ELK/OpenSearch. Datadog. New Relic. Splunk.

Documentação

C4 Model. Mermaid. PlantUML. Structurizr. Draw.io. Lucidchart. ArchiMate. ADR tools.

23. Trilha de estudos por fases

Fase 1 — Fundamentos de arquitetura

Estudar:

- O que é arquitetura de software.
- O que é arquitetura de solução.
- O que é arquitetura de integração.
- Requisitos funcionais.
- Requisitos não funcionais.
- Restrições.
- Stakeholders.
- Trade-offs.
- Diagramas básicos.
- C4 Model.
- ADR.

Prática:

- Escolher um sistema simples e criar:
- diagrama de contexto.
- diagrama de containers.
- requisitos funcionais.
- requisitos não funcionais.
- 3 ADRs.
- Fase 2 — APIs e integração síncrona

Estudar:

- HTTP.
- REST.
- OpenAPI.
- API Gateway.
- BFF.
- OAuth2/OIDC.
- Timeout.

- Retry.
- Circuit breaker.
- Rate limit.
- Idempotência.

Prática:

- Criar uma API Java/Spring com OpenAPI.
- Criar um client externo.
- Adicionar timeout, retry e circuit breaker.
- Documentar contrato.
- Criar diagrama de sequência.
- Fase 3 — Mensageria

Estudar:

- Queue.
- Topic.
- Pub/Sub.
- DLQ.
- Retry.
- Idempotent consumer.
- Poison message.
- Correlation ID.
- Message routing.
- Message transformation.

Prática:

- Criar fluxo:
- pedido criado.
- mensagem para fila.
- consumidor processa.
- falha vai para DLQ.
- reprocessamento manual.
- Fase 4 — Event-driven architecture

Estudar:

- Eventos de domínio.
- Eventos de integração.
- Kafka.
- EventBridge.
- AsyncAPI.
- CloudEvents.
- Schema Registry.
- Outbox.
- Inbox.

- Event catalog.
- Consumer lag.

Prática:

- Criar fluxo de pedidos com Kafka:
- order.created.v1
- payment.approved.v1
- invoice.issued.v1
- order.completed.v1
- Documentar eventos com AsyncAPI.
- Usar envelope CloudEvents.
- Implementar outbox.
- Implementar consumidor idempotente.
- Fase 5 — Dados e integração corporativa

Estudar:

- CDC.
- Debezium.
- ETL.
- ELT.
- Batch.
- Arquivos.
- Modelo canônico.
- Data contracts.
- Data lineage.
- Data governance.

Prática:

- PostgreSQL → Debezium → Kafka → consumidor.
- Criar processo batch com controle de lote.
- Criar relatório de rejeições.
- Fase 6 — Solução cloud

Estudar:

- Well-Architected.
- VPC/VNet.
- Containers.
- Kubernetes.
- Serverless.
- API Management.
- Event bus.
- Observabilidade.
- Segurança.
- FinOps.

- DR.

Prática:

- Desenhar solução completa em AWS ou Azure:
- frontend.
- backend.
- banco.
- cache.
- broker.
- API Gateway.
- observabilidade.
- segurança.
- CI/CD.
- rollback.
- Fase 7 — Arquitetura avançada

Estudar:

- Saga.
- CQRS.
- Event Sourcing.
- Multi-region.
- Resiliência.
- Service Mesh.
- Zero Trust.
- Multi-tenancy.
- Migração de legado.
- Strangler Fig.
- Anti-corruption layer.
- Governança.

Prática:

- Migrar módulo de um monólito para arquitetura orientada a eventos.
- Criar plano de migração incremental.
- Criar ADRs.
- Criar RFC.
- Criar plano de rollback.
- Fase 8 — Nível sênior

Estudar/praticar:

- Architecture review.
- Security review.
- Operational readiness.
- Cost review.
- Design authority.

- Mentoria.
- Comunicação executiva.
- Condução de workshops.
- Event storming.
- Trade-off analysis.
- Gestão de risco.
- Roadmap técnico.

Prática:

- Criar documento completo de arquitetura para uma plataforma real.
- Apresentar a solução como se fosse para diretoria, segurança, DevOps e desenvolvimento.
- Defender alternativas e trade-offs.

24. Projeto prático principal da trilha

Projeto: Plataforma de pedidos integrada

Desenhe e implemente uma solução com:

- Frontend.
- API Gateway.
- BFF.
- Order Service.
- Payment Service.
- Inventory Service.
- Invoice Service.
- Notification Service.
- PostgreSQL.
- Redis.
- Kafka.
- DLQ.
- Outbox.
- Inbox.
- OpenAPI.
- AsyncAPI.
- CloudEvents.
- Observabilidade.
- Segurança.
- CI/CD.
- Docker.
- Kubernetes ou ECS.
- Dashboard de métricas.
- Documentação C4.

- ADRs.
- Fluxo principal
- Cliente cria pedido.
- Order Service salva pedido.
- Outbox publica order.created.v1.
- Payment Service consome evento.
- Payment Service aprova pagamento.
- Publica payment.approved.v1.
- Inventory Service reserva estoque.
- Invoice Service emite nota.
- Notification Service envia confirmação.
- Order Service atualiza status final.
- Analytics consome eventos.
- Observabilidade rastreia tudo por correlation ID.
- Falhas obrigatórias para simular
- Pagamento indisponível.
- Estoque insuficiente.
- Nota fiscal falhando.
- Kafka fora.
- Consumidor duplicando mensagem.
- Evento inválido.
- Schema incompatível.
- Latência alta.
- DLQ acumulando.
- API externa com timeout.
- Entregáveis
- Diagrama C4 contexto.
- Diagrama C4 containers.
- Diagrama de componentes.
- Diagrama de sequência.
- Diagrama de eventos.
- OpenAPI das APIs.
- AsyncAPI dos eventos.
- ADRs.
- RFC da arquitetura.
- Plano de rollback.
- Estratégia de observabilidade.
- Estratégia de segurança.
- Estratégia de reprocessamento.
- Checklist de produção.

25. Checklist de nível sênior

Você pode considerar que está em nível sênior nessa trilha quando conseguir:

- Explicar claramente a diferença entre arquitetura de solução, software, integração, dados, cloud e enterprise.
- Levantar requisitos funcionais e não funcionais.
- Desenhar solução ponta a ponta.
- Escolher integração síncrona, assíncrona, batch, streaming ou CDC com justificativa.
- Documentar APIs com OpenAPI.
- Documentar eventos com AsyncAPI.
- Padronizar eventos com CloudEvents.
- Aplicar Enterprise Integration Patterns.
- Projetar mensageria com DLQ, retry, idempotência e reprocessamento.
- Projetar eventos com schema, versão, replay e governança.
- Aplicar Saga, Outbox, Inbox, CQRS e Event Sourcing quando fizer sentido.
- Evitar overengineering quando CRUD simples resolve.
- Projetar segurança ponta a ponta.
- Projetar observabilidade ponta a ponta.
- Definir SLOs.
- Avaliar custo.
- Avaliar risco.
- Criar ADRs e RFCs.
- Criar diagramas C4 claros.
- Defender trade-offs para público técnico e não técnico.
- Acompanhar implementação sem virar gargalo.
- Revisar arquitetura de outros times.
- Guiar desenvolvedores júnior/pleno.

26. Erros comuns que impedem senioridade

Evite:

- Escolher Kafka para tudo.
- Criar microsserviço sem necessidade.
- Usar evento sem contrato.
- Criar API sem OpenAPI.
- Criar fila sem DLQ.
- Criar retry infinito.
- Criar consumidor não idempotente.
- Ignorar timeout.
- Ignorar versionamento.

- Usar banco compartilhado como integração padrão.
- Criar arquitetura sem observabilidade.
- Criar arquitetura sem segurança.
- Criar diagrama bonito sem decisão real.
- Criar decisão técnica sem contexto.
- Confundir tecnologia com arquitetura.
- Ignorar custo.
- Ignorar operação.
- Ignorar rollback.
- Ignorar legado.
- Ignorar equipe que vai manter a solução.

27. Livros e referências recomendadas

Base obrigatória

Enterprise Integration Patterns — Gregor Hohpe e Bobby Woolf. Software Architecture: The Hard Parts — Neal Ford, Mark Richards, Pramod Sadalage, Zhamak Dehghani. Fundamentals of Software Architecture — Mark Richards e Neal Ford. Building Microservices — Sam Newman. Monolith to Microservices — Sam Newman. Designing Data-Intensive Applications — Martin Kleppmann. Release It! — Michael Nygard. Domain-Driven Design — Eric Evans. Implementing Domain-Driven Design — Vaughn Vernon. Team Topologies — Matthew Skelton e Manuel Pais.

Referências técnicas

TOGAF. AWS Well-Architected. Azure Architecture Center. Google Cloud Architecture Framework. OpenAPI Specification. AsyncAPI. CloudEvents. C4 Model. Microservices.io. Apache Kafka docs. Enterprise Integration Patterns catalog.

28. Resumo final da trilha

A trilha de Arquitetura de Integração e Arquitetura de Solução deve evoluir nesta ordem:

- Fundamentos de arquitetura
- Requisitos funcionais e não funcionais
- Diagramas e documentação
- APIs e contratos
- Mensageria
- Eventos
- Padrões de integração
- Resiliência
- Segurança
- Observabilidade
- Dados e CDC
- Cloud architecture

- Governança
- Migração de legados
- Decisão arquitetural sênior
- Comunicação e liderança técnica A diferença entre um pleno forte e um sênior nessa área não é saber citar Kafka, API Gateway ou Saga. É saber quando usar, quando não usar, quais riscos cria, como operar, como documentar, como proteger, como medir, como evoluir e como explicar para pessoas técnicas e não técnicas.